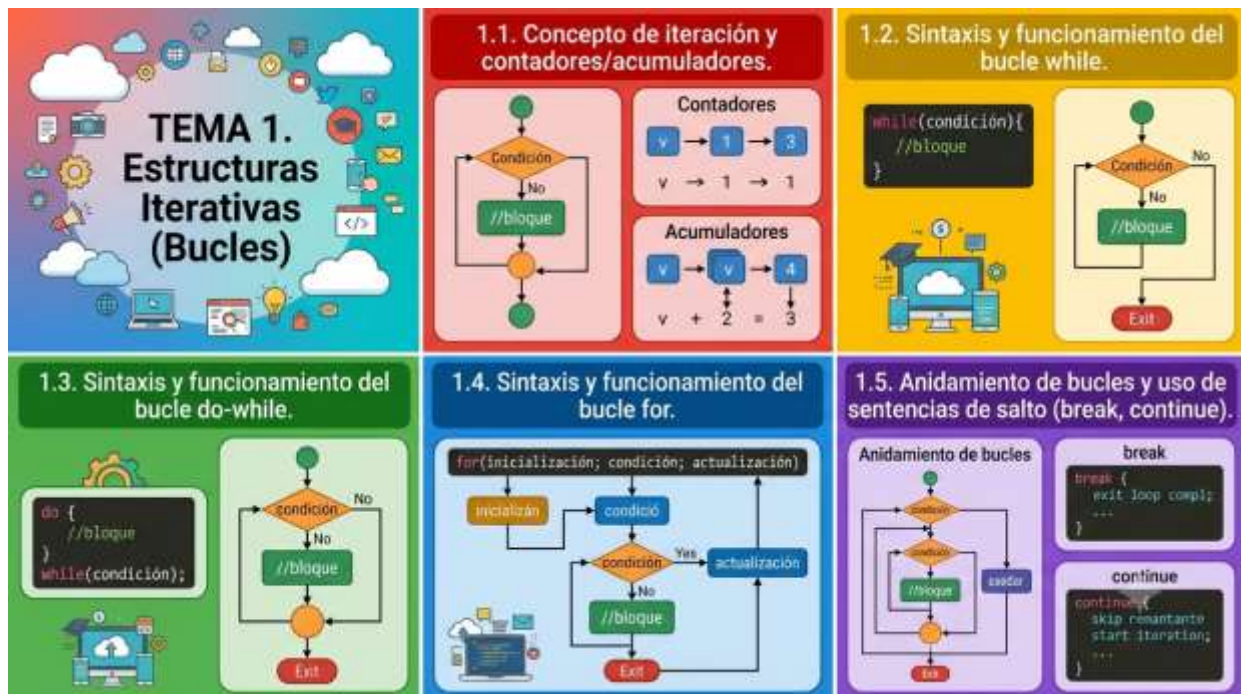


TEMA N° 1



TEMA 1. ESTRUCTURAS ITERATIVAS (BUCLES)



"EN ESTE TEMA SERÁS CAPAZ DE..."

1. **Comprender y dominar** el flujo de ejecución repetitiva en la programación, identificando cuándo y por qué es fundamental automatizar tareas utilizando bucles en lugar de escribir código redundante.
2. **Aplicar lógicamente** las diferentes estructuras iterativas (while, do-while, for) para resolver problemas reales de procesamiento de datos, gestión de información y automatización en tu entorno local.
3. **Desarrollar un pensamiento computacional eficiente**, optimizando tus algoritmos mediante el uso correcto de contadores, acumuladores y sentencias de control de flujo, como un verdadero Técnico en Sistemas Informáticos.



PRÁCTICA



¿Alguna vez te has preguntado cómo los grandes sistemas pueden procesar la información de miles de personas en segundos sin que el programador tenga que escribir la misma instrucción miles de veces?

Imagina la siguiente situación en nuestra comunidad de Pailón: El CEA Herman Ortiz Camargo acaba de inaugurar su nuevo y bien equipado módulo educativo en el Barrio Saó. Por este motivo, ha lanzado una gran campaña de inscripción gratuita para nuevos estudiantes. El éxito ha sido rotundo y, en el primer día, llegan cientos de postulantes desde diferentes puntos: comerciantes del Barrio Central Fátima, jóvenes deportistas del Barrio 24 de Septiembre y del coliseo en el Barrio 27 de Mayo, e incluso vecinos de zonas más alejadas como María Belén, San Antonio, Jardines de Pailón, Oto Waver, Ovidio Barberi, San Expedito, Las Misiones y San José.

Como el Técnico en Sistemas Informáticos encargado de la nueva sala de cómputo, se te pide crear un pequeño programa que registre el nombre de cada estudiante y a qué barrio pertenece. Tienes 500 estudiantes esperando en la fila. Si no conocieras las estructuras iterativas, tendrías que escribir la línea de código para "Registrar Estudiante" exactamente 500 veces. Esto haría que tu programa fuera gigantesco, lento, imposible de mantener y muy propenso a errores. ¿Qué pasaría si al día siguiente llegan 200 estudiantes más? Tendrías que modificar todo tu código manualmente. Debe existir una forma en la que le digas a la computadora: *"Repite esta acción de registro continuamente hasta que ya no quede nadie en la fila"*.



TEORÍA



1.1. CONCEPTO DE ITERACIÓN Y CONTADORES/ACUMULADORES

En programación, una **iteración** (comúnmente llamada "bucle" o "loop") es una estructura que nos permite repetir un bloque de código varias veces, ya sea una cantidad de veces previamente definida o hasta que se cumpla una condición específica. Esto es el corazón de la automatización.



Para controlar estas repeticiones y guardar datos mientras el bucle funciona, utilizamos dos conceptos vitales:

1. **Contador:** Es una variable que aumenta o disminuye su valor en una cantidad constante (generalmente de 1 en 1). Sirve para contar eventos. Ejemplo: Contar cuántos estudiantes del Barrio Residencial Norte se han inscrito.
2. **Acumulador:** Es una variable que incrementa o disminuye su valor en cantidades variables. Sirve para sumar totales. Ejemplo: Sumar el dinero recaudado por las ventas en la feria dominical del Barrio Central Fátima.



1.2. SINTAXIS Y FUNCIONAMIENTO DEL BUCLE WHILE

El bucle while (que en español significa "mientras") es la estructura iterativa más básica. Su lógica es simple: **"Mientras esta condición sea verdadera, sigue repitiendo este bloque de código"**.

Es ideal cuando **no sabemos exactamente cuántas veces** se va a repetir el ciclo, pero sí conocemos la condición de parada. Por ejemplo, imaginemos que estamos esperando que pase el tren en la estación del Barrio 24 de Septiembre; seguiremos esperando *mientras* el tren no haya llegado.



JavaScript

```
// Ejemplo en JavaScript: Simulando la entrada de estudiantes al CEA
// Inicializamos nuestra variable de controllet estudiantesEnFila = 5;
// Evaluamos la condición: ¿Hay estudiantes en la fila?
// Si es mayor a 0 (verdadero), entramos al bucle.while (estudiantesEnFila > 0)
{
// Esta línea se ejecuta en cada iteración    console.log("
Registrando estudiante del Barrio Saó...");
// REDUCIMOS nuestro contador de 1 en 1.    // ESTO ES CRÍTICO: Si no restamos,
el bucle será infinito.    estudiantesEnFila = estudiantesEnFila - 1;
// Mostramos cuántos quedan    console.log("
Quedan " + estudiantesEnFila + " en la fila.");
}
// Cuando estudiantesEnFila llega a 0, la condición del while es falsa y el
bucle termina.console.log("
Todos los estudiantes han sido registrados. ¡El nuevo módulo del CEA está
listo!");
```

1.3. SINTAXIS Y FUNCIONAMIENTO DEL BUCLE DO-WHILE

El bucle do-while (hacer-mientras) es un hermano muy cercano al while, pero tiene una diferencia fundamental: **primero ejecuta el bloque de código al menos una vez, y luego evalúa la condición.**

Esto es perfecto para situaciones donde necesitas garantizar que una acción ocurra obligatoriamente antes de preguntar si debe repetirse. Piensa en el sistema de validación de un cajero automático cerca del comercio formal en el Barrio Central Fátima: el sistema *debe* pedirte tu PIN al menos una vez antes de verificar si es correcto o si debe volver a pedírtelo.

JavaScript

```
// Ejemplo en JavaScript: Sistema de seguridad de la red Wi-Fi del CEA
// Declaramos la variable donde guardaremos la contraseña ingresadalet
passwordIngresada;
// Inicia el bloque '
do
' (Hacer). Esto se ejecutará sí o sí la primera vez.do {
// Simulamos que el sistema pide la contraseña (en un navegador usaríamos
prompt)    // Para este ejemplo, imaginemos que el usuario escribió "1234"
passwordIngresada = "1234";
console.log("
Intentando conectar a la red del CEA...");
// Para no hacer un bucle infinito en la prueba, cambiaremos la clave simulada
// En la vida real, aquí el usuario intentaría de nuevo.    passwordIngresada =
```




```

"
CeaHermanOrtiz2026";
// Condición: Mientras la contraseña sea diferente de la correcta, repite el '
do'}
while (passwordIngresada !== "
CeaHermanOrtiz2026");
// Si sale del bucle, significa que la contraseña fue
correcta.console.log("¡Conexión exitosa! Bienvenido a la red educativa.");

```

1.4. SINTAXIS Y FUNCIONAMIENTO DEL BUCLE FOR

El bucle for (para) es la estructura iterativa por excelencia cuando **sabemos exactamente la cantidad de veces** que queremos que se repita el código. Todo lo necesario para controlar el bucle (inicialización, condición e incremento) se agrupa en una sola línea, haciéndolo muy limpio y fácil de leer.

Imagina que el CEA adquirió exactamente 15 computadoras nuevas para el laboratorio del Barrio Saó y necesitas instalar el sistema operativo en cada una de ellas.



JavaScript

```

// Ejemplo en JavaScript: Instalación en Laboratorios
// Anatomía: for (inicialización;
condición;
incremento/decremento)
// 1. Let i = 1;

```



```

-> Empezamos en la computadora número 1.
// 2. i <= 15;
-> Repetiremos mientras i sea menor o igual a 15.
// 3. i++ -> Después de cada vuelta, sumamos 1 a la variable i (es igual a i = i + 1).for (let i = 1;
i <= 15;
i++) {
// Este código se repetirá exactamente 15 veces console.log("
Instalando software educativo en la PC número: " + i);
}
// Una vez que '
i
' llega a 16, la condición falla y salimos del bucle.console.log("
Mantenimiento finalizado. Equipos listos para los estudiantes de Sistemas
Informáticos.");

```

1.5. ANIDAMIENTO DE BUCLES Y SENTENCIAS DE SALTO (BREAK, CONTINUE)

A medida que avanzas como Técnico en Sistemas Informáticos, te enfrentarás a problemas más complejos. A veces, necesitas colocar un bucle dentro de otro bucle; a esto se le llama **anidamiento**. Además, existen palabras reservadas para alterar el flujo normal de un bucle:

1. break: "Rompe" o destruye el bucle por completo, terminándolo inmediatamente, sin importar la condición.
2. continue: Salta la iteración actual y pasa directamente a la siguiente vuelta del bucle.

Supongamos que estamos buscando el registro de un estudiante específico entre varias listas de diferentes barrios (Jardines de Pailón, Las Misiones, San José).

JavaScript

```

// Ejemplo en JavaScript: Búsqueda avanzada
// Tenemos un arreglo (lista) de barrioslet barrios = ["
Jardines de Pailón", "
Las Misiones", "
San José"];
// Supongamos que cada barrio tiene 3 estudiantes registradoslet
totalEstudiantesPorBarrio = 3;
// Bucle EXTERNO: Recorre los barriosfor (let b = 0;
b < barrios.length;
b++) {
console.log("
Revisando registros del barrio: " + barrios[b]);
// Si llegamos a San José, decidimos saltarnos este barrio por algún motivo

```



```

if (barrios[b] === "
San José") {
console.log("
Saltando este barrio por mantenimiento de datos.");
// '
continue
' detiene esta vuelta del bucle externo y pasa al siguiente barrio
continue;
}
// Bucle INTERNO: Recorre los estudiantes dentro del barrio actual    for (let e
= 1;
e <= totalEstudiantesPorBarrio;
e++) {
console.log("  -> Verificando estudiante " + e);
// Simulamos que encontramos al estudiante que buscábamos    if (barrios[b]
=== "
Las Misiones" && e === 2) {
console.log("  ¡Estudiante encontrado! Deteniendo la búsqueda completa.");
// '
break
' rompe el bucle interno.    // Nota: Para romper ambos bucles se
requiere lógica extra o etiquetas en JS.    break;
}
}
}

```

1.6. OPTIMIZACIÓN Y EL PELIGRO DE LOS BUCLES INFINITOS (TEMA AVANZADO)

Todo Técnico debe saber que un bucle mal programado puede "congelar" un sistema entero. Un **bucle infinito** ocurre cuando la condición de un bucle while o for nunca se vuelve falsa. Si en el ejemplo 1.2 olvidáramos poner la instrucción `estudiantesEnFila = estudiantesEnFila - 1;` el programa creería que siempre hay 5 estudiantes y se ejecutaría para siempre, consumiendo toda la memoria RAM de la computadora hasta que se bloquee. Escribir código no es solo hacerlo funcionar, es hacerlo eficiente.

1.7. BUCLES EN LA DOMÓTICA, EL INTERNET DE LAS COSAS (IOT) Y SENSORES (TEMA AVANZADO)

La programación iterativa cobra vida en el hardware físico. Imagina que el nuevo módulo del CEA Herman Ortiz Camargo cuenta con luces inteligentes y sensores de temperatura utilizando placas de desarrollo (microcontroladores). Estos dispositivos operan bajo un bucle principal infinito intencional (a menudo llamado `loop()` o `while(true)`). Su trabajo es leer la temperatura ambiente



miles de veces por segundo en un bucle interminable. Si detectan que el calor de Pailón supera los 35°C, el bucle activa un relé para encender los ventiladores automáticamente, y luego sigue leyendo sensores indefinidamente.



1.8. ITERACIONES EN EL ENTRENAMIENTO DE INTELIGENCIA ARTIFICIAL (TEMA AVANZADO)

Hoy en día, las estructuras iterativas son la base de la Inteligencia Artificial. Cuando un modelo de IA "aprende", lo hace mediante ciclos gigantesco llamados **épocas (epochs)**. Un modelo de aprendizaje automático pasará por miles de iteraciones procesando un conjunto de datos, comparando sus respuestas, ajustando sus errores matemáticos y volviendo a intentar. Sin el concepto fundamental de bucle for o while, el avance actual del aprendizaje profundo (Deep Learning) sería matemáticamente y operativamente imposible.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito Común	Realidad Tecnológica
------------	----------------------



"Un bucle for siempre es más rápido que un while."	El rendimiento suele ser idéntico a nivel de máquina. La elección depende de la lógica (si sabes cuántas veces iterar, usa for; si dependes de una condición dinámica, usa while).
"El anidamiento de bucles no afecta el rendimiento del programa."	Poner múltiples bucles uno dentro del otro (anidamiento profundo) multiplica drásticamente el tiempo de procesamiento. Un Técnico experto siempre busca reducir niveles de anidamiento.

□ Glosario de Términos

1. **Iteración:** Cada una de las vueltas o repeticiones individuales que realiza un bucle en su ejecución.
2. **Condición de Parada:** Expresión lógica (verdadera o falsa) que determina en qué momento un bucle debe dejar de ejecutarse.
3. **Bucle Infinito:** Error lógico en la programación donde un ciclo nunca encuentra su condición de parada, bloqueando los recursos del sistema.
4. **Acumulador:** Variable diseñada para almacenar resultados sucesivos de operaciones matemáticas dentro de un bucle, sumando o restando valores cambiantes.
5. **Hardcoding:** Mala práctica de escribir datos fijos (como escribir 500 veces una línea) en lugar de usar variables y estructuras dinámicas como bucles.

□ Ponte a Prueba

1. ¿Qué estructura iterativa garantiza que el bloque de código se ejecute obligatoriamente al menos una vez antes de evaluar cualquier condición?
 1. a) Bucle for
 2. b) Bucle while
 3. c) Bucle do-while
2. Si un programa necesita contar exactamente desde el número 1 hasta el 100, ¿cuál es el bucle más recomendado por su claridad de sintaxis?



1. a) Bucle for
2. b) Bucle while
3. c) Sentencia break

3. ¿Qué función cumple la palabra reservada continue dentro de un ciclo?

1. a) Destruye el bucle por completo.
2. b) Salta la vuelta actual del ciclo y pasa directamente a la siguiente.
3. c) Convierte un bucle normal en un bucle infinito.

(Respuestas: 1-c, 2-a, 3-b)



VALORACIÓN



Como futuros profesionales, debemos comprender que el código que escribimos tiene un impacto real en el mundo. Analicemos esto desde la perspectiva del impacto medioambiental y ético.

Cuando un Técnico en Sistemas Informáticos escribe bucles ineficientes o bucles anidados innecesarios que procesan miles de datos de manera errónea, está exigiendo que el procesador (CPU) de la computadora trabaje al máximo de su capacidad. Esto genera un alto consumo de energía eléctrica y sobrecalentamiento. A gran escala, servidores en la nube ejecutando algoritmos mal optimizados consumen enormes cantidades de kilovatios-hora, aumentando directamente la huella de carbono y afectando el medio ambiente.

Además, el estrés constante de hardware provocado por malas prácticas de programación acorta la vida útil de los equipos (baterías de laptops que se degradan rápido, procesadores quemados), lo que contribuye al alarmante problema de la "basura electrónica" (e-waste). Escribir un bucle eficiente, saber cuándo usar un break para detener búsquedas innecesarias, no es solo una cuestión de "sacar buenas notas", es una responsabilidad ética profesional para crear tecnología sustentable. Tu rol es crear soluciones informáticas para Pailón que sean rápidas, ligeras y respetuosas con los limitados recursos de hardware de las instituciones.



PRODUCCIÓN



Reto Final: Sistema Centralizado del Censo Estudiantil

Contexto: La dirección del CEA Herman Ortiz Camargo te ha solicitado digitalizar el conteo rápido de los estudiantes preinscritos que provienen de tres barrios específicos: San Expedito, María Belén y Ovidio Barberi.

Instrucciones del Laboratorio (Paso a Paso):

1. Abre tu editor de código preferido (como Visual Studio Code) o un entorno en línea como Replit o el inspector de tu navegador web.
2. Crea un archivo llamado censo_cea.js.
3. Declara tres variables que funcionarán como **acumuladores**, una para cada barrio, inicializadas en 0. (Ej: let totalMariaBelen = 0;).
4. Declara un arreglo (array) que simule una lista de ingresos en el sistema, mezclando barrios.
Por ejemplo:
let ingresos = ["María Belén", "San Expedito", "Ovidio Barberi", "María Belén", "Ovidio Barberi"];
5. Utiliza un **bucle for** para recorrer (iterar) toda la longitud de este arreglo.
6. Dentro del bucle, utiliza estructuras condicionales (if/else) para verificar qué barrio se está leyendo en cada iteración.
7. Dependiendo del barrio detectado, suma +1 al acumulador correspondiente.
8. Fuera del bucle, escribe sentencias console.log() que impriman el reporte final. Por ejemplo: "Total de estudiantes de Ovidio Barberi: 2".
9. **Desafío Extra:** Implementa un bucle while al final del código que simule una "cuenta regresiva" del 5 al 1, imprimiendo: "El sistema se cerrará en... [número]", terminando con un mensaje final de "Sistema apagado con éxito".



5. RECURSOS ADICIONALES

Para seguir puliendo tus habilidades y alcanzar la excelencia técnica, te recomiendo consultar regularmente estos recursos oficiales:

1. **MDN Web Docs (Mozilla Developer Network) - Bucles e Iteración:** La documentación más respetada y clara para entender el funcionamiento interno de las estructuras repetitivas en JavaScript.
(Búsqueda sugerida: "MDN Web Docs Loops and iteration")
2. **freeCodeCamp - Curso de Algoritmos Básicos:** Plataforma interactiva y gratuita que ofrece cientos de ejercicios prácticos directamente en el navegador, excelentes para solidificar tu lógica con bucles for y while.
(Búsqueda sugerida: "freeCodeCamp JavaScript Algorithms")
3. **W3Schools - Referencias C++ / Python / JS:** Una biblioteca enorme de ejemplos básicos y directos de sintaxis en distintos lenguajes, ideal para cuando olvides cómo se declaraba un acumulador o la condición de un bucle.
(Búsqueda sugerida: "W3Schools For Loop Tutorial")